3er Laboratorio Tecnologías de microprocesamiento

Lujan Negrin, Sury Gonzalez, Gonzalo Gorosito
Universidad Tecnológica del Uruguay, ITRSO
Fray Bentos, Uruguay
{maria.negrin.a, sury.gonzalez, gonzalo.gorosito}@estudiantes.utec.edu.uy

I. Introduccion

Los sistemas embebidos basados en microcontroladores se encuentran presentes en gran parte de los dispositivos de uso cotidiano e industrial, desde electrodomésticos hasta sistemas de climatización y automatización de edificios. En ellos, un único circuito integrado reúne la unidad de procesamiento, la memoria y un conjunto de periféricos que permiten adquirir información del entorno, procesarla y actuar sobre él en tiempo real [1].

En las instancias de laboratorio anteriores, el microcontrolador ATmega328P fue programado en lenguaje ensamblador, lo que permitió comprender en detalle su arquitectura, sus registros y el funcionamiento de sus periféricos a bajo nivel. En esta tercera instancia el desarrollo se realiza en lenguaje C, que ofrece un mayor nivel de abstracción y facilita la integración de varios periféricos en una misma aplicación, sin perder el acceso directo a los registros de configuración descritos en la hoja de datos del fabricante [2].

El presente informe documenta el diseño, la implementación y la validación de cuatro sistemas. El primero mide la temperatura ambiente mediante un sensor DHT11 y, en función de rangos definidos alrededor de un punto medio ajustable por el usuario, comanda un calefactor y un ventilador de velocidad variable. El segundo regula el nivel de iluminación sobre una superficie mediante una persiana motorizada, utilizando una fotorresistencia como sensor de luz, un potenciómetro acoplado al eje como sensor de posición y un setpoint recibido por UART. El tercero clasifica muestras de plástico según su color, iluminándolas con una tira de LEDs WS2812 y midiendo la luz transmitida con una fotocelda, para luego posicionar un servomotor sobre la bandeja correspondiente. El cuarto implementa un piano electrónico de ocho notas con dos buzzers pasivos, capaz de reproducir canciones predefinidas seleccionadas por comandos serie. Estos sistemas integran el conversor analógico-digital, la modulación por ancho de pulso, la comunicación serie, las interrupciones y la visualización en una pantalla LCD 16x2, y los datos obtenidos en los sistemas de control se registran y grafican mediante Python para analizar su respuesta.

II. Resumen

Este informe presenta el desarrollo de cuatro aplicaciones embebidas programadas en lenguaje C sobre el microcontrolador ATmega328P, con el objetivo de integrar sensores, actuadores y periféricos de comunicación de la arquitectura AVR. Se implementó un sistema de control de temperatura por rangos basado en un sensor DHT11, con punto medio ajustable por UART; un control proporcional de iluminación que acciona un motorreductor a partir de una fotorresistencia y un potenciómetro de posición; un clasificador de color de plásticos con una tira de LEDs WS2812, una fotocelda y un servomotor; y un piano electrónico de ocho notas con dos buzzers pasivos y canciones seleccionables por comandos serie. Los periféricos se configuraron directamente sobre sus registros a partir de la hoja de datos, y cada sistema se desarrolló de forma incremental, validándose en simulación con Proteus o PICSimLab antes de su montaje físico. Los datos de los sistemas de control se registraron y graficaron mediante Python para analizar su comportamiento, y el código se versionó en un repositorio de GitHub. El trabajo evidencia la importancia de ajustar el diseño teórico a las características reales de los componentes y de complementar la simulación con la validación en hardware.

***Palabras clave*—ATmega328P, lenguaje C, ADC, PWM, I²C, DHT11, LDR, WS2812, servomotor, puente H, control proporcional, sistemas embebidos.**

III. Objetivos

III-A. Objetivo general

Desarrollar en lenguaje C aplicaciones embebidas sobre el ATmega328P que integren sensores, actuadores, periféricos de comunicación e interfaces de usuario, aplicando los recursos de la arquitectura AVR a problemas de control, clasificación y generación de sonido.

III-B. Objetivos específicos

- Configurar desde C los periféricos del ATmega328P (ADC, temporizadores, USART, TWI e interrupciones) a partir de su hoja de datos, utilizando la

PWM para controlar motores, servomotores y la generación de tonos.
- Implementar la comunicación con dispositivos que requieren protocolos de temporización estricta o buses serie, como el sensor DHT11, la tira de LEDs WS2812 y la pantalla LCD mediante I^{2}C.
- Diseñar la lógica de control y las máquinas de estado de sistemas de regulación de temperatura e iluminación, clasificación de color y reproducción musical, evaluando su comportamiento mediante el registro y la graficación de datos en Python.

IV. Marco Teórico

IV-A. Conversor analógico-digital

El ADC del ATmega328P es de aproximaciones sucesivas, con 10 bits de resolución y un multiplexor que permite seleccionar entre varios canales de entrada. El resultado de una conversión simple se obtiene como

$$ADC = \frac{V_{in} \cdot 1024}{V_{ref}}, \tag{1}$$

donde V_{ref} es la tensión de referencia seleccionada. Para obtener la resolución completa, el reloj del ADC debe encontrarse entre 50 kHz y 200 kHz, por lo que se obtiene a partir del reloj del sistema mediante un preescalador. Una conversión normal requiere 13 ciclos de reloj del ADC, y la primera tras habilitar el conversor requiere 25 [2].

IV-B. Temporizadores y modulación por ancho de pulso

Los temporizadores del ATmega328P pueden operar en distintos modos según la aplicación. En el modo CTC (*Clear Timer on Compare Match*), el contador se reinicia al alcanzar el valor del registro de comparación, y si la salida se configura para conmutar en cada coincidencia se obtiene una onda cuadrada de ciclo de trabajo del 50 % con frecuencia

$$f_{CTC} = \frac{f_{clk}}{2\,N\,(1 + OCRnx)}, \tag{2}$$

donde N es el factor del preescalador [2]. Este modo resulta adecuado para la generación de tonos de frecuencia variable.

La modulación por ancho de pulso (PWM) permite controlar la potencia media entregada a una carga variando el ciclo de trabajo de una señal rectangular de frecuencia fija. En el modo Fast PWM, la frecuencia de la señal está dada por

$$f_{PWM} = \frac{f_{clk}}{N\,(1 + TOP)}, \tag{3}$$

donde TOP es el valor máximo del contador, y el ciclo de trabajo queda determinado por el valor del registro $OCRnx$ respecto de TOP [2]. En el Timer1, de 16 bits, el valor de TOP puede fijarse mediante el registro $ICR1$, lo que permite obtener con precisión frecuencias bajas como los 50 Hz requeridos por un servomotor. Aplicada a un motor de corriente continua, la PWM permite regular su velocidad, ya que la inercia mecánica y la inductancia del bobinado filtran la señal conmutada [3].

IV-C. Bus I^{2}C y pantalla LCD

El bus I^{2}C es un protocolo serie síncrono de dos líneas, SDA (datos) y SCL (reloj), ambas de colector abierto con resistencias de pull-up. Cada dispositivo esclavo se identifica mediante una dirección de 7 bits, y cada transferencia se delimita por condiciones de inicio y parada, con un bit de reconocimiento (ACK) tras cada byte [4]. En el ATmega328P este bus es implementado por el módulo TWI.

Las pantallas LCD alfanuméricas de 16x2 utilizan el controlador HD44780, que admite una interfaz paralela de 8 o 4 bits más las líneas de control RS, R/W y E [5]. Para reducir la cantidad de pines utilizados, se emplea un módulo basado en el expansor de E/S PCF8574 (Fig. 1), que recibe bytes por I^{2}C y los presenta en sus 8 salidas, operando el LCD en modo de 4 bits [6].



Figura 1: Pantalla LCD 16x2 con modulo I^{2}C basado en PCF8574

IV-D. Sensor de temperatura DHT11

El DHT11 (Fig. 2) es un sensor digital de temperatura y humedad que incluye un elemento resistivo para la humedad y un termistor NTC, junto con un microcontrolador interno que entrega una salida calibrada. Mide temperaturas entre 0 °C y 50 °C con una precisión de ±2 °C y humedad relativa entre 20 % y 90 % [7]. Se comunica mediante un protocolo propietario de un solo hilo, similar en concepto al 1-Wire pero no compatible con él: el microcontrolador mantiene la línea en nivel bajo durante al menos 18 ms como señal de inicio, el sensor responde con un pulso bajo y uno alto de 80 µs cada uno, y luego transmite 40 bits (humedad,

temperatura y suma de verificación). Cada bit comienza con un nivel bajo de 50 µs, y la duración del nivel alto siguiente determina su valor: entre 26 µs y 28 µs para un "0" y 70 µs para un "1" [7]. Dado que el rango máximo del sensor es 50 °C, el ajuste del punto medio debe restringirse para que los umbrales superiores no excedan dicho límite.

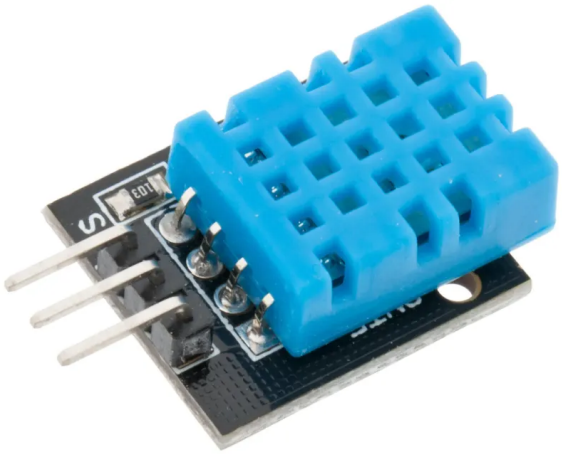


Figura 2: Sensor de temperatura y humedad DHT11

IV-E. Fotorresistencia (LDR)

Una fotorresistencia o fotocelda (Fig. 3) es un componente semiconductor, típicamente de sulfuro de cadmio, cuya resistencia disminuye al aumentar la iluminación incidente. Su relación con la intensidad luminosa es no lineal, aproximadamente de tipo potencial, y presenta tiempos de respuesta del orden de decenas de milisegundos, mayores al apagarse la luz que al encenderse [3]. Para su lectura con el ADC se conecta en un divisor resistivo, de modo que la variación de resistencia se traduce en una variación de tensión.



Figura 3: Fotorresistencia (LDR)

IV-F. Identificación de color por transmitancia

Un material translúcido coloreado transmite principalmente la luz de longitudes de onda cercanas a su propio color y absorbe las restantes. Si se ilumina una muestra sucesivamente con luz roja, verde y azul y se mide la intensidad transmitida en cada caso, se obtiene un vector de tres componentes que caracteriza su color. La transmitancia en cada canal se define como

$$T = \frac{I}{I_0}, \tag{4}$$

donde I es la intensidad medida con la muestra e I_0 sin ella. Para clasificar una muestra, el vector medido se compara con patrones de referencia obtenidos en una etapa de calibración, asignándole la clase cuyo patrón se encuentre a menor distancia, por ejemplo euclídea [8].

IV-G. Potenciómetro como sensor de posición

Un potenciómetro conectado como divisor de tensión entrega en su cursor una tensión proporcional a la posición angular de su eje,

$$V_{out} = V_{cc}\,\frac{\theta}{\theta_{max}}. \tag{5}$$

Acoplado mecánicamente al eje de un motor, permite conocer la posición real del mecanismo y detectar sus extremos de carrera.

IV-H. Motor de corriente continua y puente H

El sentido de giro de un motor de corriente continua depende de la polaridad de la tensión aplicada en sus terminales. Un puente H, formado por cuatro interruptores, permite invertir dicha polaridad desde señales lógicas.

El L298 integra dos puentes H completos capaces de manejar hasta 2 A por canal; las entradas IN1 e IN2 definen el sentido de giro y la entrada de habilitación (EN) puede recibir una señal PWM para regular la velocidad. Al estar construido con transistores bipolares, presenta una caída de tensión no despreciable entre la alimentación y el motor [9]. En este laboratorio se utiliza el módulo L298N (Fig. 4).

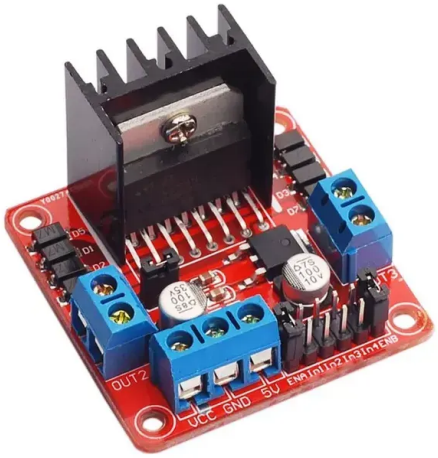


Figura 4: Modulo controlador de motores L298N

IV-I. Servomotor

Un servomotor de modelismo (Fig. 5) integra un motor de corriente continua, una caja reductora, un potenciómetro de realimentación y un circuito de control en lazo cerrado. Su posición angular se comanda mediante una señal PWM de período 20 ms (50 Hz), donde el ancho del pulso determina el ángulo: en el SG90, un pulso de aproximadamente 1 ms corresponde a un extremo, 1,5 ms a la posición central y 2 ms al extremo opuesto, para un recorrido de unos 180° [10].



Figura 5: Servomotor SG90

IV-J. LEDs direccionables WS2812

El WS2812 (Fig. 6) es un LED RGB que integra su propio circuito de control, lo que permite encadenar múltiples unidades y controlarlas individualmente con una única línea de datos. Cada LED recibe 24 bits en orden verde, rojo y azul (8 bits por color), retiene los primeros 24 bits que le llegan y retransmite el resto al siguiente LED de la cadena. La transmisión se realiza a 800 kbps, con un período de bit de 1,25 µs: un “0” se codifica con un nivel alto de 0,4 µs y un “1” con un nivel alto de 0,8 µs, con una tolerancia de ± 150 ns. Un nivel bajo sostenido por más de 50 µs indica el fin de la trama [11]. Con un reloj de 16 MHz, un período de bit equivale a solo 20 ciclos de máquina, por lo que la rutina de transmisión debe tener una temporización exacta y ejecutarse con las interrupciones deshabilitadas.



Figura 6: Tira de Leds

IV-K. *Buzzer pasivo y transistor de conmutación*

Un buzzer pasivo (Fig. 7) es un transductor electroacústico que, a diferencia del activo, no posee oscilador interno, por lo que emite un sonido de la misma frecuencia que la señal aplicada en sus terminales. Esto permite generar distintas notas musicales variando la frecuencia de una onda cuadrada producida por un temporizador. Como la corriente que puede entregar un pin del ATmega328P está limitada a un máximo absoluto de 40 mA [2], el buzzer se acciona mediante un transistor NPN 2N2222 operando en corte y saturación, con una resistencia en la base que limita la corriente de control [12].



Figura 7: Buzzer pasivo

IV-L. *Frecuencia de las notas musicales*

En la escala de temperamento igual, la octava se divide en 12 semitonos con una razón de frecuencias constante entre ellos. Tomando como referencia el La_4 de 440 Hz, la frecuencia de una nota situada n semitonos por encima o por debajo se obtiene como

$$f_n = 440 \cdot 2^{n/12} \text{ Hz}, \tag{6}$$

de donde resultan, por ejemplo, 262 Hz para el Do_4 y 523 Hz para el Do_5 [13]. Para lograr polifonía simple se utilizan dos buzzers controlados por temporizadores independientes, uno para la melodía y otro para el acompañamiento.

IV-M. *Estrategias de control*

En un sistema de control en lazo cerrado, la salida medida se compara con un valor de referencia o setpoint, y la diferencia entre ambos, denominada error, se utiliza para calcular la acción de control [14]. El control por rangos o de tipo encendido-apagado asigna una acción fija a cada intervalo de la variable medida; es sencillo de implementar y adecuado para procesos lentos como los térmicos. El control proporcional, en cambio, aplica una acción de magnitud proporcional al error,

$$u(t) = K_p\, e(t), \tag{7}$$

limitada por la saturación del actuador. En la práctica suele incorporarse una banda muerta alrededor del setpoint para evitar oscilaciones del actuador ante pequeños errores producidos por ruido o por la resolución del sensor [15].

IV-N. *Registro y visualización de datos*

Para el análisis de la respuesta de los sistemas, los datos enviados por el microcontrolador a través de la USART se reciben en la computadora mediante la biblioteca pySerial de Python [16], y se grafican con Matplotlib, una biblioteca de visualización bidimensional ampliamente utilizada en el ámbito científico [17].

V. Materiales

- Placa Arduino UNO con microcontrolador ATmega328P.
- Sensor de temperatura y humedad DHT11.
- Pantalla LCD 16x2 con controlador HD44780 y módulo $\mathrm{I^2C}$ basado en PCF8574.
- Fotorresistencias (LDR).
- Potenciómetro lineal de 10 kΩ (B10K) acoplado al eje del motorreductor.
- Motor de corriente continua (ventilador).
- Motorreductor de corriente continua tipo TT de 3 a 6 V (mecanismo de la persiana).
- Módulo controlador de motores L298N.
- Elemento calefactor o indicador de su estado.
- Tira o matriz de LEDs direccionables WS2812.
- Servomotor SG90.
- Muestras de plástico translúcido de al menos 4 colores distintos.
- 2 buzzers pasivos.
- 2 transistores NPN 2N2222.
- 8 pulsadores.
- Resistencias para divisores de tensión y polarización de transistores.
- Fuente de alimentación externa para motores y LEDs.
- Protoboard y cables de conexión.
- Computadora con Microchip Studio, Proteus, PICSimLab y Python (pySerial y Matplotlib).

VI. Procedimiento

VI-A. *Problema A*

El sistema se desarrolló de forma incremental: cada funcionalidad se implementó y verificó por separado

en simulación con Proteus antes de integrarla con las anteriores, y una vez completo se trasladó al hardware. El firmware se escribió en C con Microchip Studio para un ATmega328P a 16 MHz sobre una placa Arduino UNO.

VI-A1. Hardware: El Cuadro I resume la asignación de pines. El DHT11 se utilizó en un módulo de tres pines que incorpora la resistencia de pull-up de la línea de datos. La pantalla LCD se conectó mediante un módulo I^2C basado en el PCF8574T, cuyas líneas de dirección A0–A2 se encuentran en nivel alto, lo que corresponde a la dirección 0x27 [6]. Las líneas SDA y SCL se conectaron a los pines del módulo TWI del microcontrolador.

Cuadro I: Asignación de pines del Problema A.

Pin	Puerto	Función
0 / 1	PD0 / PD1	RX / TX de la USART
2	PD2	Datos del DHT11
4	PD4	IN2 del L298N
6	PD6	ENA del L298N (PWM, OC0A)
7	PD7	IN1 del L298N
8	PB0	LED del calefactor
A4 / A5	PC4 / PC5	SDA / SCL del módulo LCD

El ventilador se implementó con un motor de corriente continua accionado por el canal A del módulo L298N. La señal PWM se aplica a la entrada de habilitación ENA, mientras que IN1 e IN2 fijan un único sentido de giro. Durante las pruebas se comprobó que alimentar el motor desde el puerto USB, junto con el resto de los periféricos, no proporcionaba corriente suficiente, por lo que el módulo se alimentó con una fuente externa regulable, compartiendo la referencia de masa con el microcontrolador. El calefactor se representó mediante un LED con una resistencia limitadora de 220 Ω. Para las pruebas físicas se intentó inicialmente enfriar el sensor con un ventilador de 12 V, independiente del sistema de control; sin embargo, al impulsar aire a temperatura ambiente, no logró llevar el sensor por debajo de dicha temperatura, por lo que no fue suficiente para alcanzar el rango de activación del calefactor. Por este motivo, en los ensayos el enfriamiento se realizó con un equipo de aire acondicionado.

En la simulación se empleó el modelo SIMULINO UNO. La pantalla se representó mediante un PCF8574 cableado a un LCD LM016L, ya que el modelo JHD-2X16-I2C disponible en Proteus utiliza un controlador con un protocolo distinto y no responde a la dirección del PCF8574. En este caso las líneas A0–A2 se conectaron a masa, por lo que la dirección utilizada en simulación fue 0x20.

VI-A2. Arquitectura del firmware: El programa se estructuró como un bucle principal que consulta banderas activadas por interrupciones. Las rutinas de servicio de interrupción realizan únicamente operaciones breves: el Timer1 marca la base de tiempo y la USART almacena los caracteres recibidos. Las tareas de mayor duración, como la lectura del sensor, la actualización de los actuadores y el procesamiento de comandos, se ejecutan en el bucle principal. De esta forma ninguna tarea extensa bloquea las interrupciones y el sistema puede atender comandos en cualquier momento. Las variables compartidas entre las interrupciones y el bucle principal se declararon como `volatile`.

VI-A3. Temporización y lectura del sensor: Las mediciones se realizan cada 5 s utilizando el Timer1 en modo CTC con un preescalador de 1024 y $\mathrm{OCR1A} = 15624$, lo que produce una interrupción por segundo:

$$f = \frac{16\,\mathrm{MHz}}{1024\,(1 + 15624)} = 1\,\mathrm{Hz}. \tag{8}$$

Un período de 5 s requeriría 78 125 cuentas, valor que excede la capacidad de 16 bits del temporizador, por lo que la rutina de interrupción cuenta cinco eventos antes de activar la bandera de medición.

La lectura del DHT11 implementa el protocolo descrito en [7]. Tras la señal de inicio, el programa espera cada flanco con un tiempo límite, de modo que una falla o desconexión del sensor produce un código de error en lugar de bloquear el programa. El valor de cada bit se determina muestreando la línea 40 μs después del flanco de subida: si permanece en nivel alto, el bit es un "1". Durante la recepción de la trama, de aproximadamente 4 ms, las interrupciones se deshabilitan para no alterar la medición de tiempos, guardando y restaurando el registro `SREG`. Finalmente se verifica la suma de comprobación y, si la lectura es inválida, el sistema mantiene la última acción aplicada.

VI-A4. Control por rangos: Los umbrales de control no se definieron como valores fijos, sino como distancias $d = T - T_{pm}$ entre la temperatura medida T y el punto medio T_{pm}, según el Cuadro II. Con el punto medio por defecto de 21 °C se obtienen los rangos especificados en el enunciado, y al modificarlo todos los umbrales se desplazan en conjunto, cumpliendo el requisito de recalibración automática.

Cuadro II: Rangos de control en función de la distancia al punto medio.

d (°C)	T con $T_{pm} = 21$	Acción
≤ -6	0–15	Calefactor encendido
-5 a 4	16–25	Calefactor y ventilador apagados
5 a 14	26–35	Ventilador baja (40 %)
15 a 24	36–45	Ventilador media (70 %)
≥ 25	≥ 46	Ventilador alta (100 %)

La velocidad del ventilador se controla con el Timer0 en modo Fast PWM sobre la salida OC0A, con un preescalador de 8, lo que resulta en una frecuencia de $16\,\mathrm{MHz}/(8 \cdot 256) \approx 7{,}8$ kHz. Esta frecuencia se encuentra por encima del rango en el que la conmutación resulta

audible en el motor y es compatible con la velocidad de conmutación del L298N [9]. Para el apagado del motor se desconecta la salida OC0A del pin y se fuerza su nivel a cero, ya que en modo Fast PWM no invertido un valor de $\mathrm{OCR0A} = 0$ genera igualmente un pulso de un ciclo de reloj en cada período [2].

Una única función aplica la acción correspondiente a la temperatura medida sobre el motor, el calefactor, la pantalla LCD y la línea de datos serie, y es invocada tanto en cada medición como al modificar el punto medio.

VI-A5. Ajuste del punto medio: El menú interactivo por UART se implementó como la máquina de estados de la Fig. 8, con los estados *inactivo*, *esperando opción* y *esperando valor*. Desde el estado inactivo, el comando `M` despliega las opciones: modificar el punto medio, consultar el estado actual o salir. Mientras el menú permanece abierto, el control continúa operando y solo se suspenden los mensajes de monitoreo, para que no se mezclen con las opciones del menú.

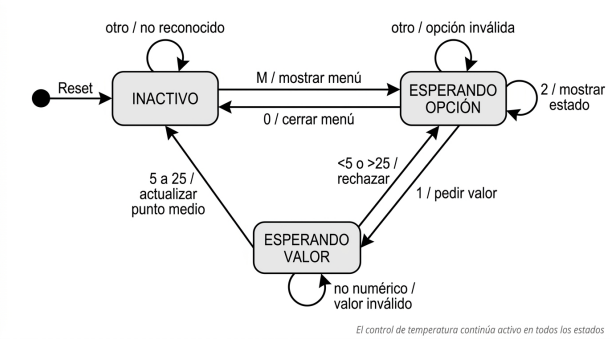


Figura 8: Máquina de estados del menú de ajuste del punto medio.

El valor ingresado se valida antes de aplicarse. Dado que la ventilación alta comienza en $T_{pm} + 25$ °C y el rango máximo del DHT11 es 50 °C [7], el punto medio se limitó a un máximo de 25 °C; un valor superior es rechazado, informando al usuario el motivo y conservando el valor anterior. Este límite se calcula en el código a partir del máximo del sensor, por lo que un cambio de sensor solo requiere modificar una constante. De forma análoga, se fijó un mínimo de 5 °C para que el rango de confort no comience por debajo de 0 °C. Al aceptarse un nuevo valor, la acción se recalcula de inmediato con la última temperatura válida, sin forzar una nueva lectura, dado que el DHT11 requiere al menos 1 s entre mediciones.

VI-A6. Interfaz LCD: La pantalla se controló con la biblioteca provista por los docentes, que gestiona el PCF8574 mediante el módulo TWI y opera el controlador HD44780 en modo de 4 bits [5]. Durante las pruebas se observó que se perdía el primer carácter escrito después de borrar la pantalla. La causa es que el comando de borrado requiere aproximadamente 1,6 ms de ejecución [5], mientras que la biblioteca, con el bus a 400 kHz, enviaba el siguiente dato antes de ese tiempo. Se corrigió agregando un retardo de 2 ms en la función de borrado. La primera línea de la pantalla muestra la temperatura y la acción vigente, y la segunda el rango de confort definido por el punto medio.

VI-A7. Registro de datos: En cada aplicación del control, el microcontrolador envía por la USART, a 9600 baudios, una línea con el formato `D,<temp>,<pm>,<calef>,<vel>`, cuyo prefijo permite distinguirla de los mensajes del menú. En la computadora, un programa en Python basado en pySerial [16] utiliza un hilo dedicado a la lectura del puerto, que almacena los datos en un archivo CSV junto con el instante de recepción, mientras el hilo principal envía al microcontrolador los comandos ingresados por teclado. Esto permite operar el menú sin interrumpir el registro. Al finalizar, los datos se grafican con Matplotlib [17], mostrando la evolución de la temperatura junto con la banda de confort, la velocidad del ventilador y el estado del calefactor.

VI-A8. Control de versiones: El código se versionó en un repositorio público de GitHub, cuyo enlace se incluye en el Anexo A, registrando cada etapa del desarrollo incremental en un commit independiente: lectura del sensor y visualización en el LCD, comunicación UART, base de tiempo con el Timer1, control del motor por PWM, control por rangos, salida del calefactor, menú de ajuste del punto medio y registro de datos en Python.

VI-B. Problema B

El sistema se desarrolló con la misma metodología incremental que el Problema A: cada funcionalidad se implementó y verificó por separado en simulación con Proteus antes de integrarla con las anteriores, y una vez completo se trasladó al hardware. El firmware se escribió en C con Microchip Studio para un ATmega328P a 16 MHz sobre una placa Arduino UNO.

El enunciado plantea una persiana motorizada que regula la luz incidente sobre una superficie. En el prototipo de laboratorio, el mecanismo se representó mediante un motorreductor de corriente continua acoplado directamente al eje de un potenciómetro, de modo que la posición angular del eje equivale a la apertura de la persiana: el giro en sentido horario corresponde a abrirla y el antihorario, a cerrarla.

VI-B1. Hardware: El Cuadro III resume la asignación de pines. La LDR se conectó en un divisor resistivo con una resistencia fija de 10 kΩ hacia masa, de forma que un aumento de la iluminación reduce la resistencia de la LDR y eleva la tensión en la entrada analógica. Para medir la posición se utilizó un potenciómetro lineal (curva B) de 10 kΩ, con sus extremos conectados a 5 V

y a masa y el cursor a la entrada ADC1. Su impedancia de salida, de 2,5 kΩ en el peor caso, se encuentra dentro del máximo de 10 kΩ recomendado para las entradas analógicas [2].

Cuadro III: Asignación de pines del Problema B.

Pin	Puerto	Función
0 / 1	PD0 / PD1	RX / TX de la USART
6	PD6	ENA del L298N (PWM, OC0A)
7	PD7	IN1 del L298N
8	PB0	IN2 del L298N
A0	PC0	Divisor de la LDR (ADC0)
A1	PC1	Cursor del potenciómetro (ADC1)

El motor se accionó con el canal A del módulo L298N. La señal PWM se aplica a la entrada ENA, para lo cual se retiró el puente que la mantiene habilitada, mientras que IN1 e IN2 determinan el sentido de giro. Se utilizó un motorreductor de tipo TT, de 3 a 6 V nominales. Con la fuente utilizada inicialmente, el motor giraba demasiado rápido para el recorrido del potenciómetro, de aproximadamente 270°, por lo que el módulo se alimentó con una fuente de 6,3 V. Considerando la caída de tensión de aproximadamente 2 V del L298N [9], la tensión máxima aplicada al motor resulta cercana a 4,3 V. La fuente comparte la referencia de masa con el microcontrolador.

En la simulación se empleó el modelo SIMULINO UNO, una LDR con iluminación ajustable, un potenciómetro interactivo, el integrado L298 con sus entradas de sensado de corriente conectadas a masa y un motor de corriente continua. Dado que en Proteus no existe acoplamiento mecánico entre el motor, el potenciómetro y la iluminación, ambos sensores se variaron manualmente para emular el efecto del mecanismo y verificar la respuesta del microcontrolador.

VI-B2. Arquitectura del firmware: Al igual que en el Problema A, el programa se estructuró como un bucle principal que consulta banderas activadas por interrupciones. La interrupción del Timer1 marca la base de tiempo y la de recepción de la USART almacena los caracteres del setpoint; la adquisición, el cálculo del control, el accionamiento del motor y el envío de datos se ejecutan en el bucle principal.

VI-B3. Temporización y adquisición: El lazo de control se ejecuta cada 10 ms, utilizando el Timer1 en modo CTC con un preescalador de 64 y $\mathrm{OCR1A} = 2499$:

$$f = \frac{16\,\mathrm{MHz}}{64\,(1 + 2499)} = 100\,\mathrm{Hz}. \tag{9}$$

En cada período se leen secuencialmente los canales ADC0 y ADC1. El ADC utiliza AVCC como referencia y un preescalador de 128, lo que da un reloj de conversión de 125 kHz, dentro del rango de 50 a 200 kHz en el que se obtiene la resolución completa de 10 bits [2]. Además, se deshabilitaron los buffers digitales de ambas entradas mediante el registro `DIDR0` para reducir el consumo y el ruido. La lectura N de la LDR se convierte a porcentaje, $L = \mathrm{round}(100\,N/1023)$, para expresarla en las mismas unidades que el setpoint.

VI-B4. Recepción del setpoint: El setpoint se envía por la USART como un número entero entre 0 y 100, que representa el porcentaje de iluminación deseado. La rutina de interrupción de recepción acumula los caracteres en un buffer hasta recibir un fin de línea, momento en el que activa una bandera; el bucle principal valida entonces que la cadena contenga solo dígitos y que el valor no supere 100. Si el valor es válido, se actualiza el setpoint y se confirma el cambio; en caso contrario, se informa el error y se conserva el valor anterior.

VI-B5. Lazo de control: El controlador implementado es proporcional, con banda muerta y límites en la acción de control. A partir del error $e = SP - L$, el valor de PWM aplicado se calcula como:

$$\mathrm{PWM} = \begin{cases} 0 & \text{si } |e| \le 2, \\ \min\left(80 + 8\,|e|,\ 150\right) & \text{si } |e| > 2, \end{cases} \tag{10}$$

mientras que el signo del error define el sentido de giro: un error positivo indica falta de luz, por lo que el motor gira en sentido horario para abrir la persiana; un error negativo indica exceso de luz, por lo que gira en sentido antihorario para cerrarla. La señal PWM se genera con el Timer0 en modo Fast PWM sobre la salida OC0A, con un preescalador de 64, lo que resulta en una frecuencia de $16\,\mathrm{MHz}/(64 \cdot 256) \approx 976$ Hz.

El diagrama de flujo del enunciado se tomó como base, con cuatro modificaciones. En primer lugar, en la rama de error negativo el diagrama asigna directamente $\mathrm{PWM} = e$, un valor negativo que el registro `OCR0A` no admite, por lo que se utilizó la magnitud del error y su signo se reservó para el sentido de giro. En segundo lugar, el diagrama solo detiene el motor cuando el error es exactamente cero, condición que en la práctica no se alcanza debido a las variaciones de la lectura de la LDR; sin una banda muerta, el motor oscilaría permanentemente alrededor del setpoint, por lo que se incorporó una banda de $\pm 2\,\%$. En tercer lugar, se agregó un PWM mínimo de 80, ya que con ciclos de trabajo bajos el motor no logra vencer su fricción estática. Por último, el error se calcula en porcentaje y se escala mediante una ganancia de 8, ya que asignar directamente el error en cuentas del ADC, de hasta 1023, saturaría la salida ante cualquier error significativo y anularía el comportamiento proporcional.

El límite superior de 150 se incorporó al pasar al hardware, con el fin de reducir la velocidad máxima del motor y evitar que, debido a la inercia, superara los límites de carrera antes de detenerse.

VI-B6. Límites de carrera y frenado: El potenciómetro, además de informar la posición del mecanismo, define los límites de carrera de la persiana. El accionamiento del motor se concentra en una única función que, antes de aplicar el comando, bloquea el giro horario si la lectura alcanzó el límite superior y el antihorario si alcanzó el inferior. Solo se bloquea el sentido que empuja contra el límite, de modo que el mecanismo siempre puede alejarse de él. En la simulación se utilizaron límites de 50 y 973 cuentas; al acoplar el motor directamente al potenciómetro, se ampliaron a 150 y 873, lo que deja un margen aproximado de 40° hasta cada tope mecánico. Antes de cerrar el lazo se verificó que la lectura del potenciómetro aumentara con el giro horario, condición necesaria para que los límites actúen sobre el sentido correcto.

Para detener el motor, en lugar de deshabilitar el puente, se mantiene ENA en nivel alto con IN1 e IN2 en nivel bajo. En esta condición el L298N cortocircuita los terminales del motor y produce un frenado rápido [9], lo que reduce el giro por inercia tras alcanzar un límite o entrar en la banda muerta.

VI-B7. Máquina de estados: El comportamiento del sistema se resume en la máquina de estados de la Fig. 9. Tras la inicialización de los periféricos, el sistema permanece en uno de tres estados, *detenido*, *abriendo* o *cerrando*, y las transiciones entre ellos se evalúan cada 10 ms a partir del error y de la posición del potenciómetro. La recepción de un nuevo setpoint puede ocurrir en cualquier estado y modifica el error que determina la siguiente transición.

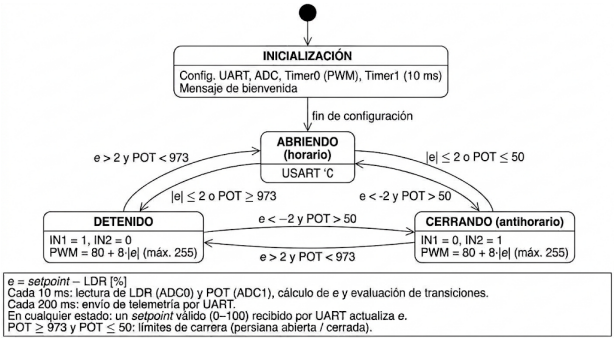


Figura 9: Máquina de estados del control de iluminación.

VI-B8. Registro de datos: Cada 200 ms, el microcontrolador envía por la USART, a 9600 baudios, una línea con el siguiente formato:

```
SP=<sp>,LDR=<ldr>,POT=<pot>,
    PWM=<pwm>,DIR=<H|A|S>
```

que contiene el setpoint, la iluminación medida, la posición, el PWM aplicado y el sentido de giro. Dado que la transmisión de cada línea demora aproximadamente 40 ms, durante ese intervalo se omiten algunas iteraciones del lazo, en las que el motor mantiene el último comando aplicado; el efecto sobre el control resulta despreciable.

En la computadora, un programa en Python basado en pySerial [16] y Matplotlib [17] grafica los datos en tiempo real en tres paneles: iluminación medida y setpoint, posición del mecanismo con los límites de carrera, y PWM con signo según el sentido de giro. Cada muestra se almacena en un archivo CSV junto con el instante de recepción. Como el puerto serie solo puede ser abierto por un programa a la vez, un hilo secundario lee los setpoints ingresados por teclado y los envía al microcontrolador, de modo que la misma aplicación permite operar y monitorear el sistema.

VI-B9. Control de versiones: El código se versionó en el repositorio público indicado en el Anexo A, registrando cada etapa del desarrollo en un commit independiente: recepción y validación del setpoint por UART, lectura de los sensores por ADC con envío periódico de datos, control del motor por PWM con sentido de giro y límites de carrera, lazo de control proporcional con banda muerta y, finalmente, frenado activo, PWM máximo y límites de seguridad para el acople directo al potenciómetro.

VI-C. Problema C

El sistema se desarrolló de forma incremental: cada funcionalidad se implementó y verificó por separado en simulación antes de integrarse con el hardware físico. El firmware se escribió en lenguaje C para un microcontrolador ATmega328P operando a un reloj de 16 MHz.

VI-C1. Hardware y asignación de periféricos: El diseño integra los siguientes módulos conectados a los pines del microcontrolador, resumidos en la siguiente tabla

Cuadro IV: Asignación de pines y puertos del Arduino.

Pin del Arduino	Puerto AVR	Función
D0/D1	PD0/PD1	RX y TX de la USART para terminal virtual
D6	PD6	Datos de la matriz LED WS2812
D9	PB1(OC1A)	Salida Fast PWM para el control del servomotor
A0	PC0(ADC0)	Entrada analógica para la fotorresistencia (LDR)
A4/A5	PC4/PC5	Líneas SDA y SCL del bus TWI para la pantalla LCD

VI-C2. Entorno de simulación y compilación condicional: La validación virtual del bucle principal, la lectura analógica y la interfaz visual se llevó a cabo utilizando el entorno de simulación PICSimLab. Para solventar las discrepancias en el orden de los canales de color de la matriz LED entre el entorno gráfico del simulador y el circuito real, se implementó una directiva de preprocesamiento condicional (`MODO_SIMULACION`). Al encontrarse activa, configura la rutina de transmisión con el estándar GRB de la simulación; al comentarse, conmuta automáticamente al orden de la tira física. Cabe

destacar que, si bien la hoja de datos del WS2812B define teóricamente el orden GRB, la tira física empleada operó correctamente con el orden BRG, sugiriendo el uso de un controlador alternativo compatible, lo cual se gestionó limpiamente mediante dicha directiva.

VI-C3. Adquisición de señales y justificación del método: Para mitigar el ruido eléctrico y las fluctuaciones de la fotorresistencia, la lectura analógica se optimizó mediante una función que promedia 10 lecturas consecutivas (`ADC_Read_Smooth`). Este procedimiento corresponde a un promedio aritmético de muestras para filtrado de ruido y no a una técnica de sobremuestreo estricta (la cual requeriría un incremento de resolución mediante decimación).

Si bien el marco teórico de clasificación de color por transmitancia describe la obtención de un vector de tres componentes iluminando secuencialmente con luz roja, verde y azul y calculando distancias euclídeas respecto a patrones de referencia, en esta implementación práctica se simplificó el proceso evaluando una única lectura analógica de la fotorresistencia frente a umbrales de tensión fijos. Esta simplificación se adoptó debido al alcance del montaje experimental y a la estabilidad suficiente obtenida con iluminación fija para diferenciar las muestras ensayadas.

VI-C4. Control de actuadores (servomotor y matriz WS2812): **Matriz LED WS2812:** Debido a los estrictos requerimientos temporales del protocolo (pulsos de ancho fijo a 800 kbps), la función de control se programó utilizando bloques de ensamblador inline dentro de C, asegurando la sincronización exacta de los ciclos de instrucción con el reloj del sistema.

Servomotor: Se configuró el Timer1 en modo Fast PWM con TOP definido en el registro ICR1 a una frecuencia de 50 Hz (período de 20 ms), ajustando el registro de comparación OCR1A sobre el pin digital D9 para posicionar mecánicamente el eje hacia la bandeja correspondiente según el color detectado.

VI-C5. Interfaz LCD y gestión de caracteres residuales: La visualización se gestionó mediante una librería basada en TWI y el expansor PCF8574 operando el controlador HD44780 en modo de 4 bits. Durante las pruebas de actualización dinámica, se evidenció la persistencia de caracteres fantasma debido a variaciones en la longitud de las cadenas de texto. Esto se solucionó aplicando un formateo de ancho fijo en las funciones de impresión, lo que garantiza el rellenado automático con espacios en blanco y limpia los residuos visuales en pantalla.

VI-C6. Arquitectura de control y máquina de estados: El flujo operativo del firmware se estructuró formalmente a través de la máquina de estados. Tras la inicialización de los periféricos y un retardo de estabilización de un segundo, el sistema transita al estado base o de reposo (Apagado). A partir de allí, se ejecuta un bucle principal de evaluación continua (while(1)), encargado de realizar la lectura suavizada de la fotorresistencia y mapear el valor analógico (`ADC_Raw`). Dependiendo del rango de tensión detectado, el flujo conmuta de manera determinista hacia los estados correspondientes: Amarillo (145–168, servo a 45°), Verde (210–250, servo a 90°), Azul (300–340, servo a 75°), Rojo (370–435, servo a 30°) y el estado de reposo / Apagado (con valor por defecto 0–0 ante valores fuera de rango), actualizando los actuadores y retornando de forma controlada al ciclo de evaluación o al estado base.

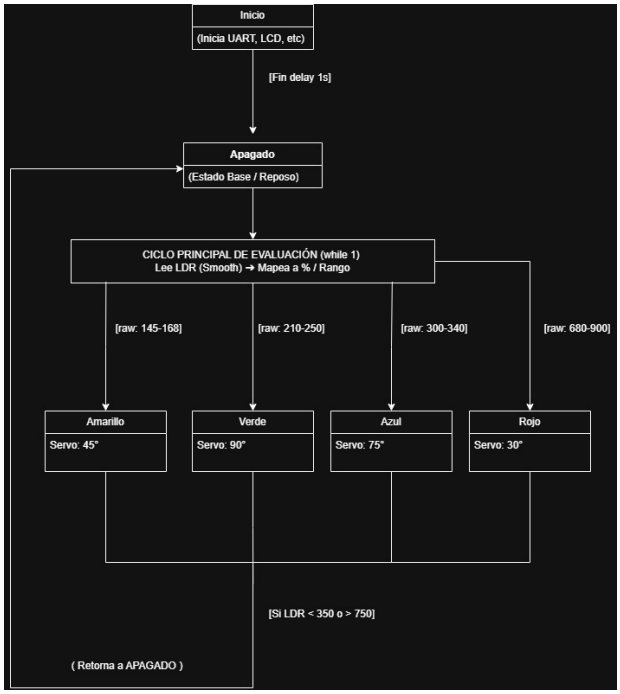


Figura 10: Maquina de estados del sistema clasificador de color de plasticos

VI-C7. Control de versiones: El código se versionó en un repositorio público de GitHub, registrando el desarrollo incremental en seis commits sucesivos: (1) inicialización de periféricos base; (2) implementación de la función driver WS2812 en C con soporte condicional; (3) estructuración del bucle principal y transmisión serial; (4) definición de los rangos de calibración de lectura ADC; (5) integración de actuadores (servomotor y matriz LED); y (6) optimización del formateo de ancho fijo en la LCD.

VI-D. Problema D

El piano se desarrolló de forma incremental: primero se generó una sola nota con un temporizador, luego se agregaron las ocho teclas, la base de tiempo, el segundo buzzer, el reproductor de canciones, la comunicación

serie y, por último, la pantalla LCD. Cada etapa se verificó en simulación (Proteus) antes de pasar al montaje físico. El programa se escribió en C con Microchip Studio para un ATmega328P a 16 MHz sobre una placa Arduino UNO.

VI-D1. Hardware: El Cuadro V resume la asignación de pines. Las teclas se conectaron entre cada pin y masa, con la resistencia de pull-up interna habilitada. En Proteus se colocaron ocho pulsadores conectados a esos pines, y la misma asignación se usó en el montaje físico. Los pines PC4 y PC5 se reservaron para el bus I^2C y PD0 y PD1 para la USART.

Cuadro V: Asignación de pines del Problema D.

Pin	Puerto	Función
A0–A3	PC0–PC3	Teclas 1 a 4 (Do, Re, Mi, Fa)
2 / 3	PD2 / PD3	Teclas 5 y 6 (Sol, La)
8 / 10	PB0 / PB2	Teclas 7 y 8 (Si, Do agudo)
9	PB1 (OC1A)	Buzzer de melodía (Timer1)
11	PB3 (OC2A)	Buzzer de bajo (Timer2)
0 / 1	PD0 / PD1	RX / TX de la USART
A4 / A5	PC4 / PC5	SDA / SCL del módulo LCD

La consigna indica accionar los buzzers pasivos mediante un transistor. En la simulación se incluyó un transistor NPN 2N2222 por buzzer (Q1 y Q2), en corte y saturación: cada buzzer se conectó entre la alimentación y el colector, el emisor a masa y la base a la salida del temporizador (OC1A y OC2A) a través de una resistencia limitadora de 1 kΩ. Con una alimentación de 5 V y una tensión base-emisor de aproximadamente 0,7 V, la corriente de base resulta de unos 4,3 mA, muy inferior a los 40 mA por pin que admite el microcontrolador. Las líneas SDA y SCL del bus I^2C se llevaron a la alimentación con resistencias de pull-up de 4,7 kΩ. En el montaje físico, en cambio, los buzzers se conectaron directamente entre los pines OC1A y OC2A y masa, sin resistencia en serie y sin el transistor. La pantalla es un LCD 16x2 con módulo I^2C basado en el PCF8574T, con dirección 0x27 en el módulo físico.

VI-D2. Conexión directa de los buzzers: En el montaje físico los buzzers se conectaron directamente a los pines OC1A y OC2A, sin transistor, y con esa configuración las ocho notas y las tres canciones se reprodujeron correctamente en las pruebas realizadas. Dado que toda la corriente del buzzer circula por el pin del microcontrolador, cuyo máximo absoluto es de 40 mA según la hoja de datos, y que no se midió la corriente consumida, no se verificó que se mantenga dentro de ese límite. En la simulación sí se utilizó el transistor 2N2222 indicado en la consigna.

VI-D3. Generación de las notas: Las frecuencias de las ocho teclas corresponden a la escala de Do mayor, desde el Do_4 (262 Hz) hasta el Do_5 (523 Hz), y se obtienen de la ecuación (6). Para la melodía y el modo piano se utilizó el Timer1 en modo Fast PWM con TOP definido por el registro ICR1 (modo 14) y un preescalador $N = 8$. Despejando TOP de la ecuación (3) se obtiene

$$\mathrm{ICR1} = \frac{f_{clk}}{N\,f} - 1, \qquad \mathrm{OCR1A} = \frac{\mathrm{ICR1}}{2}, \tag{11}$$

donde la segunda expresión fija un ciclo de trabajo del 50 %, que es el que maximiza la potencia del tono en un buzzer pasivo. Con $N = 8$ la nota más grave (262 Hz) requiere $\mathrm{ICR1} = 7632$, valor que cabe en los 16 bits del registro, por lo que no fue necesario cambiar de preescalador.

Para el acompañamiento se utilizó el Timer2 en modo CTC con conmutación de la salida OC2A y $N = 256$. De la ecuación (2) resulta

$$\mathrm{OCR2A} = \frac{f_{clk}}{2\,N\,f} - 1. \tag{12}$$

Al ser un registro de 8 bits, el menor valor de frecuencia posible con este preescalador es $16\,\mathrm{MHz}/(2\cdot256\cdot256) \approx 122$ Hz, por debajo de la nota más grave usada en los bajos (La_3, 220 Hz, con $\mathrm{OCR2A} = 141$). El valor se trunca a un entero, lo que introduce un error de frecuencia que se cuantifica en la sección de resultados.

VI-D4. Base de tiempo y antirrebote: Para no usar retardos bloqueantes se configuró el Timer0 en modo CTC con preescalador $N = 64$ y $\mathrm{OCR0A} = 249$, generando una interrupción en cada coincidencia de comparación. La frecuencia de interrupción resulta

$$f_{int} = \frac{f_{clk}}{N\,(1 + \mathrm{OCR0A})} = \frac{16\,\mathrm{MHz}}{64 \cdot 250} = 1\,\mathrm{kHz}, \tag{13}$$

es decir, una interrupción cada 1 ms. La rutina de servicio solo incrementa un contador, y la función `millis()` lo devuelve al programa principal. Con esa referencia el antirrebote se resuelve sin esperas: cada 10 ms se lee el estado de las teclas y se considera válido un cambio cuando dos lecturas consecutivas coinciden.

Al encender, el programa lee el nivel de reposo de los pines de las teclas y detecta una pulsación como un cambio respecto de ese nivel (operación XOR), por lo que el resultado no depende de la polaridad del conexionado. Esto exige que ninguna tecla esté apretada al encender o reiniciar.

VI-D5. Reproductor de canciones: Cada canción se guarda en memoria flash (`PROGMEM`) como dos tablas, una para la melodía y otra para el bajo, cuyos elementos contienen la frecuencia y la duración de la nota. Una frecuencia nula representa un silencio. Las duraciones se expresan en unidades relativas, donde 1 es la nota más corta de la partitura, y se multiplican por una constante de tempo propia de cada canción: 150 ms para The Sims, 250 ms para FNAF y 300 ms para He's a Pirate.

Las melodías se transcribieron a mano a partir de tutoriales en video con piano animado: para cada nota

se anotó su nombre y su duración relativa, las teclas negras se escribieron como la tecla blanca anterior con sostenido y la octava se eligió de modo de minimizar los saltos. Como el teclado físico solo cubre ocho notas, las canciones usan además notas fuera de esa escala, que se generan igualmente porque el reproductor trabaja con la frecuencia y no con la tecla. El bajo se ubicó en el tiempo siguiendo las indicaciones de la partitura, y cuando debe entrar después que la melodía se antepone un silencio.

Cada voz (melodía y bajo) es una estructura que guarda su tabla, la posición actual y el instante en que termina la nota en curso. En cada vuelta del bucle principal se invoca una función que apaga la nota si venció su duración y, si la nota anterior terminó, carga la siguiente. Dos decisiones evitan errores de temporización:

- El instante de fin se calcula sobre una línea de tiempo teórica (`t_fin += dur`) y no sobre el instante real de la lectura, de modo que el retraso de una nota no se acumula en las siguientes.
- Los últimos 60 ms de cada nota se dejan en silencio, para que dos notas iguales consecutivas se distingan como notas separadas.

Como la función no espera, la canción avanza a la vez que se atiende la comunicación serie y puede detenerse en cualquier momento.

VI-D6. Comunicación serie: La USART se configuró a 9600 baudios, 8 bits de datos, sin paridad y un bit de parada. Con $f_{clk} = 16$ MHz el registro de velocidad es $\mathrm{UBRR} = f_{clk}/(16 \cdot \mathrm{baud}) - 1 = 103$, que produce 9615 baudios reales, un error del 0,16 %. La recepción se consulta sin esperar: si hay un carácter disponible se lee y, si no, el programa continúa. Los comandos son `1`, `2` y `3` para iniciar The Sims, FNAF y He's a Pirate, respectivamente, y `S` o `s` para detener la reproducción. Al arrancar, el sistema envía por la USART el menú de comandos y, durante el funcionamiento, informa cada cambio de modo.

VI-D7. Interfaz LCD: La pantalla se controló con la biblioteca provista por los docentes (`twi.c`, `twi.h` y `twi_lcd.h`). Como la función de inicialización de la biblioteca modifica todo el puerto C, que contiene cuatro de las teclas, no se utilizó: el módulo TWI se configuró directamente con $\mathrm{TWBR} = 24$ y preescalador 1, lo que da un bus de $16\,\mathrm{MHz}/(16+2\cdot24) = 250$ kHz. Dado que el módulo físico respondió en 0x27 pero el componente de Proteus lo hizo en 0x3F, el programa explora las direcciones 0x20 a 0x3F con un tiempo límite por intento, usa la primera que responde e informa el resultado por la USART. Si ninguna responde, el piano sigue funcionando sin pantalla.

La pantalla muestra, según el estado, un mensaje de bienvenida, el nombre de la nota que se está tocando o el nombre de la canción en reproducción.

VI-D8. Máquina de estados: El programa se organizó como la máquina de estados de la Fig. 11, con tres estados. En ST_INICIO la pantalla muestra el mensaje de bienvenida y suena un pitido de arranque de 300 ms en La; al cumplirse 1500 ms el sistema pasa a ST_PIANO. En ST_PIANO cada tecla suena mientras permanece apretada y la pantalla muestra su nota. Los comandos `1`, `2` o `3` cargan e inician una canción y llevan a ST_CANCION, donde se reproducen la melodía y el bajo y la pantalla muestra el nombre de la canción. Desde allí, el comando `S` o el fin de la canción detienen los buzzers y vuelven a ST_PIANO, mientras que un nuevo comando `1`, `2` o `3` reinicia la reproducción con otra canción.

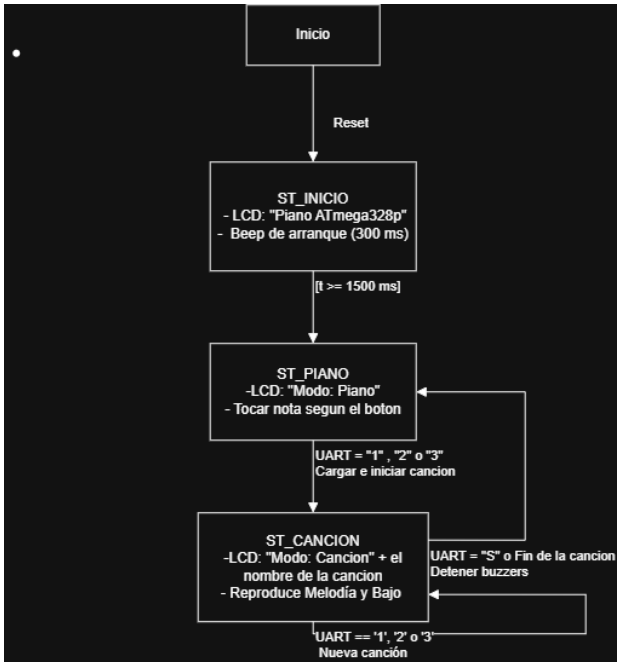


Figura 11: Máquina de estados del piano

VI-D9. Control de versiones: El código se versionó en el repositorio del Anexo A, registrando el desarrollo en seis commits sucesivos: (1) definición de las frecuencias, las estructuras y la base de datos de canciones; (2) incorporación del Timer0 para `millis()` y del control PWM y CTC de los dos buzzers; (3) comunicación UART para consola y depuración; (4) configuración del bus I^2C (TWI) y de las funciones de control del LCD; (5) lectura de los botones físicos con el sistema antirrebote; y (6) motor de voces, máquina de estados y función `main`.

VII. Resultados

VII-A. Problema A

El sistema se verificó primero en simulación y luego en el montaje físico, donde se realizaron varias pruebas que confirmaron el funcionamiento de todas las acciones de control, del menú de ajuste y del registro de datos. A

continuación se presentan los resultados más representativos.

VII-A1. Simulación: En Proteus se recorrieron todos los rangos de control modificando la temperatura del modelo del DHT11, incluyendo los valores límite entre tramos (15/16, 25/26, 35/36 y 45/46 °C), y en todos los casos la acción aplicada coincidió con el Cuadro II. La Fig. 12 muestra el caso de 15 °C con el punto medio por defecto: la pantalla indica la acción `CALEF`, el terminal virtual informa la activación del calefactor y el LED correspondiente se enciende, mientras el motor permanece detenido.

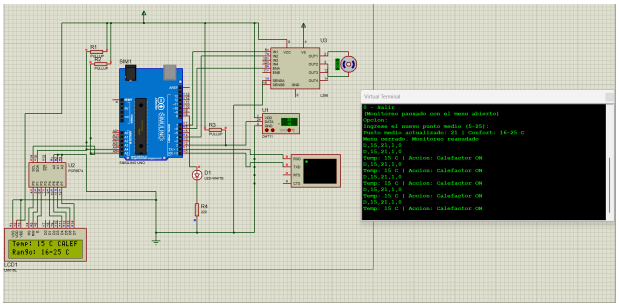


Figura 12: Simulación en Proteus con 15 °C y punto medio de 21 °C: activación del calefactor.

El menú de ajuste se verificó tanto en simulación como en el hardware. Los valores dentro del rango permitido se aplicaron de inmediato, actualizando el rango mostrado en la pantalla y la acción vigente sin esperar a la siguiente medición. Los valores superiores a 25 fueron rechazados con un mensaje que indica la temperatura a la que comenzaría la ventilación alta y el máximo del sensor, los valores inferiores a 5 fueron rechazados por dejar el rango de confort por debajo de 0 °C, y las entradas no numéricas fueron descartadas solicitando nuevamente el valor. En todos los casos de rechazo se conservó el punto medio anterior.

VII-A2. Implementación física: La Fig. 13 muestra el montaje final. El motor se alimentó con una fuente de 12 V a través del L298N, con masa común al microcontrolador, y se conservaron los ciclos de trabajo de 40, 70 y 100 %, con los que el motor arrancó y las tres velocidades resultaron claramente distinguibles.

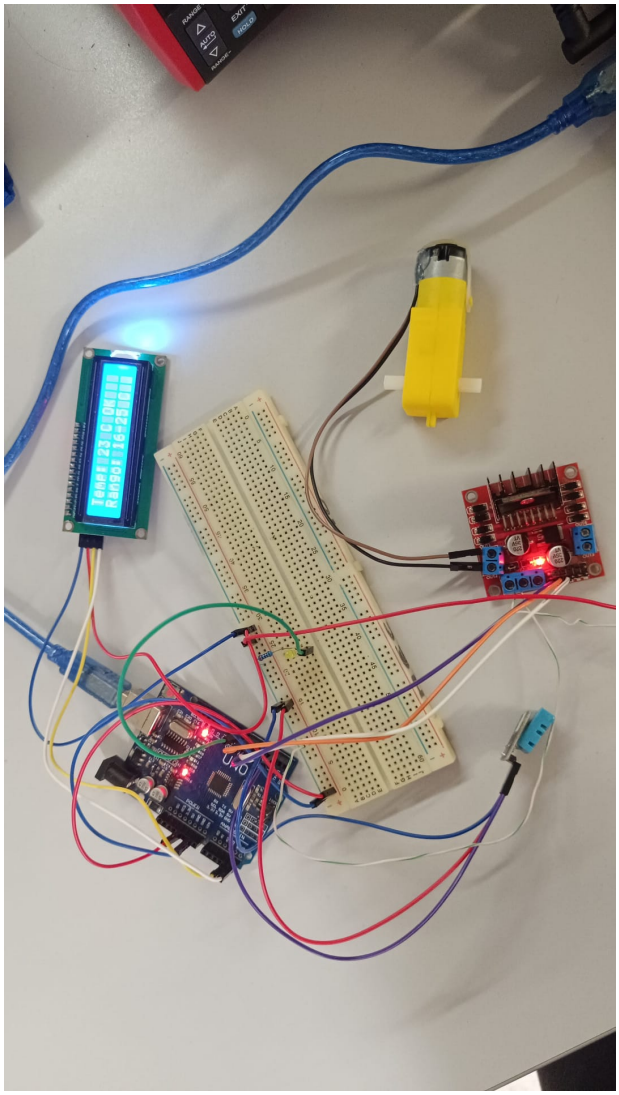


Figura 13: Montaje físico del sistema de control de temperatura.

VII-A3. Ensayo con registro de datos: La Fig. 14 presenta una corrida de aproximadamente 17 minutos registrada con el programa en Python. Dado que la temperatura ambiente del laboratorio permanecía dentro del rango de confort, el ensayo combinó la modificación del punto medio con la variación física de la temperatura del sensor, enfriándolo con un equipo de aire acondicionado y calentándolo con una fuente de calor puntual (un encendedor de chispa por fricción).

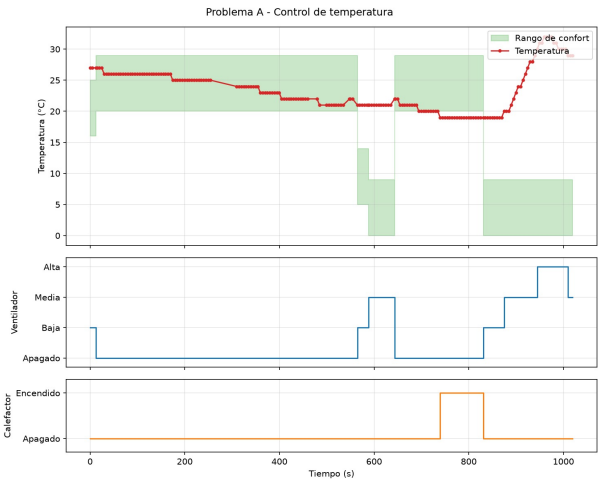


Figura 14: Evolución de la temperatura, el rango de confort y las acciones de control durante el ensayo.

El ensayo comenzó con 27 °C y el punto medio por defecto, por lo que el ventilador arrancó en velocidad baja. Al elevar el punto medio a 25 °C, el rango de confort pasó a 20–29 °C y el motor se detuvo en el mismo instante. Durante los minutos siguientes el sensor se enfrió progresivamente de 27 a 21 °C sin que el sistema actuara, por encontrarse dentro del rango de confort. Cerca de los 565 s el punto medio se redujo a 10 y luego a 5 °C, y el ventilador pasó a velocidad baja y media respectivamente, mostrando la recalibración de los rangos con una misma temperatura. Al restablecer el punto medio en 25 °C, el motor volvió a detenerse.

A los 740 s, aproximadamente, la temperatura descendió a 19 °C, que con un punto medio de 25 °C corresponde a $d = -6$, y el calefactor se encendió. Se mantuvo activo hasta que el punto medio se redujo a 5 °C, momento en que se apagó y el ventilador pasó a velocidad baja. Finalmente, al calentar el sensor, la temperatura subió de 19 a 32 °C y el ventilador pasó a velocidad media a los 20 °C ($d = 15$) y a velocidad alta a los 30 °C ($d = 25$). Al disminuir la temperatura a 29 °C ($d = 24$), el sistema regresó a velocidad media.

En todas las transiciones el cambio de acción se produjo exactamente en los umbrales definidos. La temperatura varía en escalones de 1 °C, que corresponden a la resolución del DHT11, y los intervalos sin datos en la gráfica corresponden a los períodos en que el menú permaneció abierto, durante los cuales el envío de datos se suspende mientras el control continúa operando.

VII-A4. Diferencias entre simulación y hardware: Durante el desarrollo se identificaron las siguientes diferencias:

- **Modelo de LCD en Proteus:** el componente JHD-2X16-I2C no respondió a la dirección del PCF8574, lo que se comprobó con el depurador I^2C del simulador, por lo que se reemplazó por un PCF8574 y un LM016L cableados, equivalentes al módulo físico.
- **Retardo del comando de borrado:** la pérdida del primer carácter tras borrar la pantalla, causada por la falta de espera en la biblioteca, se corrigió con un retardo de 2 ms.
- **Corriente del puerto USB:** alimentado desde el Arduino, el motor no recibía corriente suficiente al compartir la alimentación con el resto de los periféricos, por lo que se utilizó una fuente externa.
- **Caída de tensión del L298N:** al tratarse de un driver bipolar, el L298N presenta una caída de aproximadamente 2 V [9] que no se reproduce en el modelo ideal de la simulación, donde el motor gira a su velocidad nominal.
- **Iluminación de fondo del LCD:** el módulo físico requirió colocar el puente de la luz de fondo, aspecto inexistente en la simulación.

VII-B. Problema B

VII-B1. Simulación: La Fig. 15 muestra el circuito simulado en Proteus. Cada etapa del desarrollo se verificó a través del terminal virtual antes de integrar la siguiente.

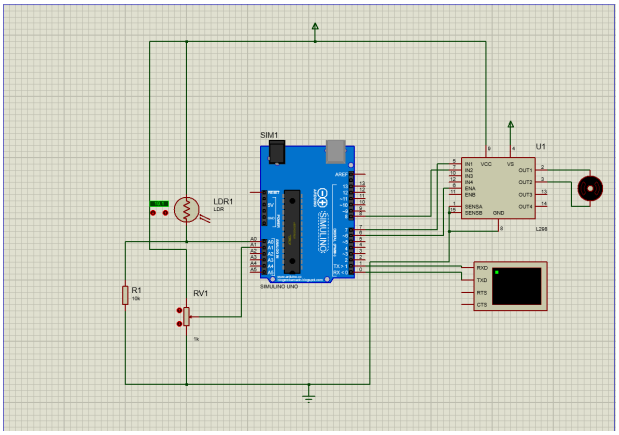


Figura 15: Circuito del Problema B simulado en Proteus.

La recepción del setpoint aceptó los valores 0, 75 y 100, y rechazó con un mensaje de error las entradas fuera de rango (101), las negativas (-5) y las no numéricas (`abc`), conservando en todos los casos el valor anterior.

En la verificación del ADC, la lectura del potenciómetro recorrió de 1 a 1023 cuentas; el valor mínimo distinto de cero se debe a que el control interactivo del modelo no lleva el cursor exactamente al extremo. La lectura de la LDR pasó de 1 % con 0,1 lux a 80 % con 100 lux, y por encima de ese valor aumentó cada vez más lentamente. Este comportamiento refleja la relación no lineal entre la resistencia de la LDR y la iluminancia, que en el divisor resistivo produce una respuesta aproximadamente

logarítmica, y limita el rango de setpoints alcanzables en la simulación.

El control del motor se verificó primero en lazo abierto, asignando el setpoint directamente a la velocidad y al sentido de giro. Se comprobó que el motor se detenía al alcanzar cada límite de carrera y que, estando en un límite, arrancaba al solicitar el sentido opuesto.

Finalmente, se verificó el lazo de control con un setpoint de 50 %, variando manualmente la iluminación de la LDR. La Tabla **??** compara los valores de PWM obtenidos con los calculados a partir de la ley de control; en esta etapa, previa a la incorporación del PWM máximo, la saturación se encontraba en 255. La Fig. 16 muestra la salida del terminal durante esta prueba, en la que el PWM disminuye a medida que la iluminación se acerca al setpoint. Al entrar en la banda muerta, el motor se detuvo, y con iluminaciones inferiores a 48 % invirtió su sentido de giro.

Cuadro VI: Verificación del lazo de control en simulación ($SP = 50\,\%$).

LDR (%)	e (%)	PWM calc.	PWM obt.	Sentido
71	-21	248	248	A
69	-19	232	232	A
63	-13	184	184	A
59	-9	152	152	A
56	-6	128	128	A
54	-4	112	112	A

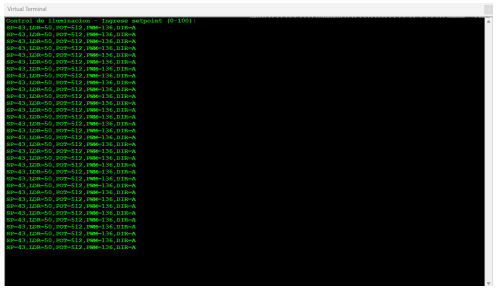


Figura 16: Salida del terminal virtual durante la verificación del lazo de control.

VII-B2. Ensayo en hardware: La Fig. 17 presenta el registro de un ensayo realizado durante la evaluación del laboratorio, de 288,6 s de duración y 1313 muestras, con un setpoint fijo de 43 %. A pedido de los docentes, en este ensayo se deshabilitó la banda muerta, de modo que el motor solo se detenía con un error nulo. Durante la primera parte del ensayo, el motor permaneció acoplado al potenciómetro; posteriormente se retiró el acople y la posición y la iluminación se variaron manualmente, para observar la respuesta del controlador ante distintas combinaciones de las entradas. La iluminación se modificó cubriendo la LDR, con lo que su lectura recorrió de 0 % a 87 %.

Durante los primeros 24 s, la iluminación medida fue de 74 %, lo que corresponde a un error de $-31\,\%$ y requeriría cerrar la persiana; sin embargo, el potenciómetro se encontraba en 81 cuentas, por debajo del límite inferior, por lo que el motor permaneció detenido. Al cubrir la LDR, el error pasó a ser positivo y el motor giró en sentido horario con un PWM que aumentó de 96 a 150 a medida que crecía el error, llevando el mecanismo desde 89 cuentas hasta superar el límite superior en aproximadamente 2,2 s. A partir de ese momento el motor se detuvo, aunque por inercia el eje continuó girando hasta 937 cuentas, lo que representa un sobrepaso de 64 cuentas, equivalente a unos 17°, dentro del margen previsto hasta el tope del potenciómetro. De forma análoga, entre los 47 y 58 s el exceso de luz hizo girar el motor en sentido antihorario hasta superar el límite inferior, con un sobrepaso máximo de 92 cuentas, equivalente a unos 24°, y entre los 74 y 80 s la falta de luz lo llevó nuevamente hasta el límite superior.

Entre los 80 y 150 s, la LDR permaneció cubierta, con un error de aproximadamente 41 %, mientras que el mecanismo se encontraba más allá del límite superior; el motor se mantuvo detenido durante todo ese intervalo. A partir de los 150 s, con el acople retirado, cada vez que el potenciómetro se llevó por debajo del límite superior el motor volvió a girar con el PWM máximo, y se detuvo cada vez que la lectura superó nuevamente las 873 cuentas. Entre los 210 y 240 s, la iluminación se varió rápidamente por encima y por debajo del setpoint, y el sentido de giro siguió en cada muestra el signo del error.

Entre los 240 y 268 s, la iluminación se mantuvo entre 36 % y 50 %, en torno al setpoint. En este tramo el PWM operó en la zona proporcional, entre 88 y 136, sin alcanzar la saturación. Al no existir banda muerta, las pequeñas variaciones de la lectura de la LDR alrededor del setpoint invirtieron repetidamente el sentido de giro, y el motor se detuvo únicamente en las muestras con error nulo; en 42 muestras del ensayo, el motor fue accionado con errores de 1 o 2 %, que la banda muerta de $\pm 2\,\%$ habría anulado. Este comportamiento confirma la necesidad de la banda muerta incorporada al diseño. Finalmente, a partir de los 268 s, la iluminación aumentó hasta cerca de 70 % y el motor giró en sentido antihorario con el PWM saturado.

El análisis del archivo CSV del ensayo mostró que no hubo ninguna muestra en la que el motor fuera accionado hacia un límite de carrera ya alcanzado, y que el PWM y el sentido de giro registrados coinciden en todas las muestras con los calculados a partir de la ley de control. El período medio entre muestras fue de 0,22 s, levemente superior a los 200 ms programados, lo que concuerda con el tiempo de transmisión de cada línea estimado en la metodología.

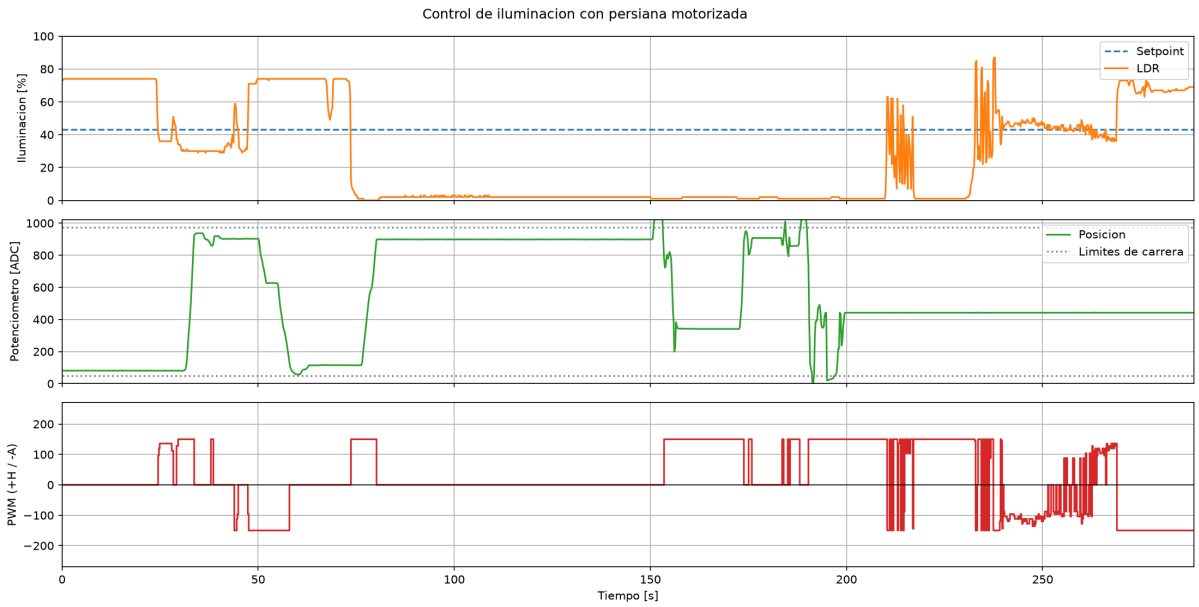


Figura 17: Gráfica resultante del ensayo en hardware: iluminación y setpoint (arriba), posición del potenciómetro (centro) y PWM con signo según el sentido de giro (abajo).

Cabe señalar que, en el prototipo, el movimiento del motor no modifica la iluminación que recibe la LDR, por lo que el lazo no se cierra físicamente y el ensayo verifica la respuesta del controlador ante las variaciones de sus entradas. Por otra parte, dado que la LDR mide la luz total incidente, el control no puede compensar variaciones de la iluminación ajenas al mecanismo regulado, tal como se plantea en las limitaciones del enunciado.

VII-C. Problema C

El sistema se verificó primero en simulación y luego en el montaje físico, donde se evaluó el rendimiento de la lectura analógica, la detección de muestras, el posicionamiento del servomotor y la interfaz visual.

VII-C1. Simulación: En el entorno de simulación PICSimLab se validó el comportamiento del bucle principal y la directiva #define `MODO_SIMULACION`. Se recorrieron los distintos rangos de lectura del conversor analógico-digital (`ADC_Raw`), comprobando que cada intervalo correspondía de manera determinista al color esperado y posicionando el servomotor mediante el pin D9 según la lógica de la máquina de estados.

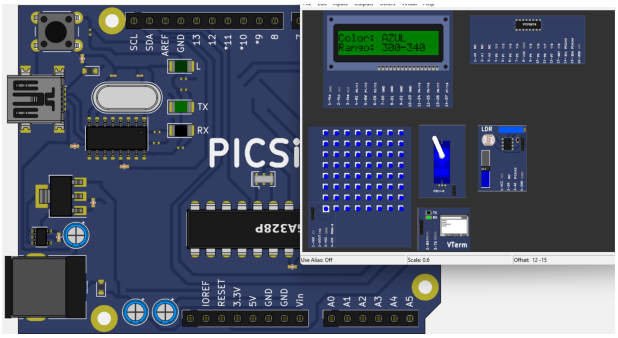


Figura 18: Simulación del sistema clasificador de color de plásticos

VII-C2. Implementación física y validación experimental: En el montaje físico se desactivó el modo de simulación, adoptando el orden de canales BRG. Para respaldar cuantitativamente el funcionamiento del sistema y reflejar la dispersión observada en los ensayos experimentales, se realizaron pruebas repetitivas con N=20 intentos por cada categoría de color, registrando el valor promedio en bruto del conversor analógico-digital (`ADC_Raw`), sus valores extremos (mínimo y máximo) y el porcentaje de aciertos en la clasificación (Tabla VII).

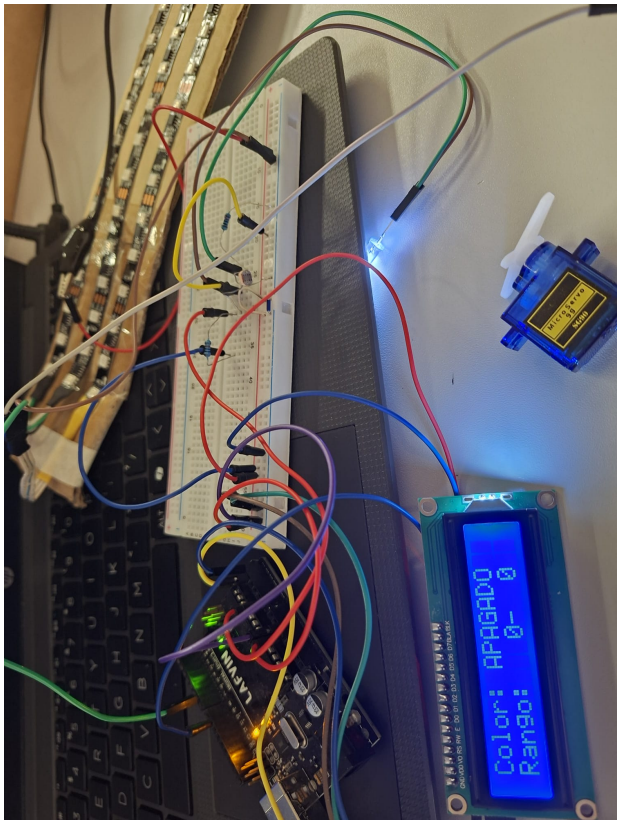


Figura 19: Montaje físico del sistema clasificador de color de plásticos

Cuadro VII: Resultados detallados de medición y clasificación por color.

Color	Rango ADC Esperado	Lectura Min	Lectura ADC Promedio	Lectura Max	Ángulo Servo Objetivo	Intentos(N)	Aciertos	Porcentaje de éxito
Amarillo	145–168	148	156	165	45°	20	18	90 %
Verde	210–250	215	230	246	90°	20	16	80 %
Azul	300–340	305	321	338	75°	20	16	80 %
Rojo	370–435	380	402	425	30°	20	19	95 %

Como se observa en el TablaVII, las muestras verde y azul presentaron una mayor dispersión en los valores extremos medidos y exigieron mayor ajuste experimental durante las pruebas. En contraste, el estado rojo exhibió una detección inmediata y muy estable. A pesar de estas variaciones, el filtrado por promedio de 10 muestras (`ADC_Read_Smooth`) permitió mantener tasas de éxito de al menos 80 % en todas las categorías.

VII-C3. Diferencias entre simulación y hardware: Durante la transición de la simulación al montaje físico se identificaron las siguientes diferencias:

- **Orden de canales RGB en la matriz LED:** La simulación requirió el estándar GRB, mientras que el hardware real operó correctamente con BRG debido a diferencias en el controlador de la tira física.
- **Fluctuaciones analógicas en la LDR:** En la simulación los valores del ADC fueron perfectamente estáticos, mientras que en el montaje físico el ruido eléctrico y la luz ambiental afectaron principalmente a las muestras verde y azul, requiriendo el promedio de 10 muestras (`ADC_Read_Smooth`) para mitigar falsos cambios de estado.
- **Alimentación de potencia:** La corriente combinada de la matriz LED y el servomotor exigió el uso de una fuente externa independiente para evitar caídas de tensión que afectaran la estabilidad del microcontrolador ATmega328P

VII-D. Problema D

El sistema se verificó primero en simulación y luego en el montaje físico, donde se confirmó el funcionamiento de las ocho teclas, de las tres canciones, de los comandos por la USART y de la pantalla.

VII-D1. Simulación: En Proteus (Fig. 20) se comprobó que cada una de las ocho teclas genera su nota, que la pantalla muestra el nombre correspondiente y que los comandos `1`, `2`, `3` y `S` inician y detienen las canciones. Para reproducir los tonos se utilizó el componente SOUNDER, ya que el componente BUZZER ACTIVE no sigue la frecuencia de entrada, y cada uno se accionó con un transistor 2N2222 como indica la consigna. La pantalla se representó con el componente I^2C-20X4, que respondió en la dirección 0x3F; el escaneo del programa la encontró sin modificar el código. La terminal virtual de la USART mostró el menú de arranque y los mensajes de cambio de modo.

Aun con el transistor, el sonido de la simulación se escuchó entrecortado. Proteus advirtió que la simulación no se ejecutaba en tiempo real por la carga de CPU, y el mismo programa sonó correctamente en el montaje físico, por lo que se atribuye el efecto al simulador y no a la lógica del programa.

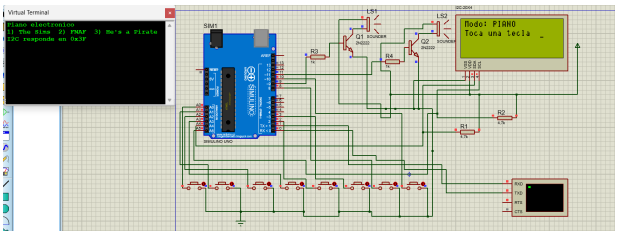


Figura 20: Simulación en Proteus del piano

VII-D2. Implementación física: La Fig. 21 muestra el montaje final. Los buzzers pasivos se conectaron directamente a los pines OC1A y OC2A, sin transistor ni resistencia en serie (ver la conexión directa en el Procedimiento). Con esa conexión las ocho notas sonaron correctamente mientras la tecla permanecía apretada, y al soltarla el sonido se detenía. Las tres canciones se reprodujeron a dos voces con la melodía en un buzzer y el bajo en el otro. La pantalla, que respondió en 0x27,

mostró el nombre de cada nota y el de la canción en curso.

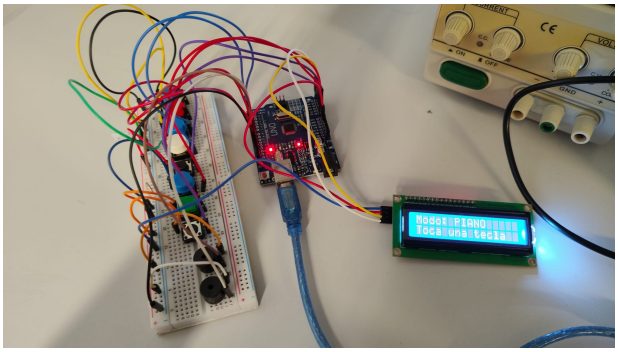


Figura 21: Montaje físico del piano

En la prueba física la canción de FNAF sonaba demasiado rápida, por lo que se ajustó su constante de tempo (actualmente 250 ms por unidad) y se revisaron las duraciones de sus notas.

VII-D3. Frecuencias generadas: La Tabla VIII compara la frecuencia deseada de cada tecla con la que resulta de los valores enteros de los registros del Timer1, calculada con la ecuación (3). El error es menor al 0,02 % en todas las notas, muy por debajo de un semitono (alrededor de 5,9 %), por lo que no es audible. En los bajos, generados con el Timer2, el error máximo es de 0,91 % (en el Sol_4, 392 Hz, con $\text{OCR2A} = 78$), también inferior a un semitono, aunque mayor porque el registro de 8 bits tiene menos resolución.

Cuadro VIII: Valores teóricos de los registros del Timer1 ($N = 8$) para las notas del piano.

Nota	f (Hz)	ICR1	f_{real} (Hz)	Error (%)
Do	262	7632	262,02	0,008
Re	294	6801	294,03	0,011
Mi	330	6059	330,03	0,010
Fa	349	5729	349,04	0,012
Sol	392	5101	392,00	0,001
La	440	4544	440,04	0,010
Si	494	4047	494,07	0,014
Do agudo	523	3823	523,01	0,002

VII-D4. Diferencias entre simulación y hardware: Durante el desarrollo se identificaron las siguientes diferencias:

- **Transistor de los buzzers:** la simulación incluyó un 2N2222 por buzzer, como pide la consigna, mientras que el montaje físico los conectó directamente a los pines del microcontrolador. Ambas configuraciones reprodujeron las notas.
- **Sonido entrecortado en la simulación:** el simulador no corrió en tiempo real por la carga de CPU y las notas se escucharon trabadas, mientras que en el montaje físico las canciones sonaron con la temporización esperada.
- **Polaridad de las teclas:** en una de las pruebas de simulación el conexionado de los pulsadores no coincidía con el supuesto y el buzzer sonaba de forma continua. Se resolvió leyendo el nivel de reposo al arrancar.
- **Tipo de buzzer en Proteus:** el componente BUZZER ACTIVE emite un tono fijo y no sigue la frecuencia del temporizador, por lo que se reemplazó por SOUNDER.
- **Dirección I^2C de la pantalla:** 0x3F en Proteus y 0x27 en el módulo físico. Se resolvió con el escaneo automático de direcciones.
- **Inicialización de la biblioteca:** la función `twi_init()` sobrescribía todo el puerto C y las teclas dejaban de funcionar, por lo que se configuró el TWI directamente.
- **Enlace del compilador:** el uso de `memcpy_P` produjo un error de enlace en Microchip Studio, y se reemplazó por la lectura directa de flash con `pgm_read_word`.

VIII. Conclusiones

Se cumplió el objetivo general de desarrollar en lenguaje C aplicaciones embebidas sobre el ATmega328P que integran sensores, actuadores, periféricos de comunicación e interfaces de usuario, aplicadas a problemas de control, clasificación y generación de sonido. El paso de ensamblador a C permitió abordar sistemas de mayor complejidad sin perder el control sobre el hardware, y la validación de cada sistema primero en simulación y luego en el montaje físico confirmó su funcionamiento en condiciones reales.

En cuanto a la configuración de los periféricos, el ADC, los temporizadores, la USART, el módulo TWI y las interrupciones se configuraron directamente sobre sus registros a partir de la hoja de datos. Esto permitió calcular de antemano los parámetros de funcionamiento, como frecuencias de PWM, bases de tiempo, velocidades de comunicación y relojes de conversión, y contrastarlos luego con el comportamiento medido. La PWM demostró su versatilidad al emplearse con distintos fines: regular la velocidad de motores de corriente continua, posicionar un servomotor y generar tonos de frecuencia variable, en este último caso con errores de frecuencia inferiores a un semitono.

Respecto a la comunicación con dispositivos externos, se implementaron protocolos de temporización estricta, como el del sensor DHT11 y el de la tira WS2812, que exigieron controlar los tiempos a nivel de microsegundos y deshabilitar las interrupciones durante las transferencias, y se utilizó el bus I^2C para controlar la pantalla LCD con solo dos líneas. La experiencia mostró que la documentación teórica no siempre coincide con el hardware disponible: las direcciones I^2C de los módulos

y el orden de colores de la tira de LEDs debieron determinarse experimentalmente.

En relación con el diseño de la lógica de control, las estrategias por rangos y proporcional, junto con las máquinas de estados, resultaron adecuadas para organizar sistemas con varios modos de operación. Estructurar los programas de forma no bloqueante, con bases de tiempo generadas por temporizadores y banderas activadas por interrupciones, permitió atender en forma concurrente los comandos serie, la pantalla y los actuadores. El registro de datos y su graficación en Python fueron determinantes para evaluar los sistemas, ya que permitieron verificar las transiciones de control y justificar con evidencia decisiones de diseño como la banda muerta, los límites de PWM y el promediado de lecturas, necesarias ante la no linealidad de los sensores, la inercia de los motores y el ruido del entorno.

Como líneas de mejora, se plantea cerrar físicamente los lazos de control con mecanismos reales, implementar métodos de medición más robustos frente a las condiciones del entorno y respetar en todos los casos las etapas de potencia y los límites eléctricos del microcontrolador, verificando mediante mediciones las corrientes involucradas.

APÉNDICE A
REPOSITORIO

El código fuente desarrollado en este laboratorio, junto con el historial de versiones de cada problema, se encuentra disponible en el siguiente repositorio público de GitHub:

https://github.com/gonzalogorosito-collab/Labs-MIcro-2026

Los videos que demuestran el funcionamiento de los sistemas desarrollados en este laboratorio se encuentran disponibles en la siguiente carpeta de Google Drive:

https://drive.google.com/drive/folders/1woMGeHJ_09q5Esdnf7QmbBJxEBlVcoOq

REFERENCIAS

[1] M. A. Mazidi, S. Naimi, and S. Naimi, *The AVR Microcontroller and Embedded Systems: Using Assembly and C*. Upper Saddle River, NJ: Prentice Hall, 2011.

[2] Microchip Technology Inc., *ATmega48A/PA/88A/PA/168A/PA/328/P megaAVR Data Sheet*, Microchip Technology Inc., 2020, dS40002061B.

[3] P. Horowitz and W. Hill, *The Art of Electronics*, 3rd ed. Cambridge: Cambridge University Press, 2015.

[4] NXP Semiconductors, *UM10204: I^2C-bus specification and user manual*, rev. 7.0 ed., NXP Semiconductors, 2021.

[5] Hitachi Ltd., *HD44780U (LCD-II) Dot Matrix Liquid Crystal Display Controller/Driver*, Hitachi Ltd., 1998, aDE-207-272(Z).

[6] Texas Instruments, *PCF8574 Remote 8-Bit I/O Expander for I^2C Bus*, Texas Instruments.

[7] D-Robotics UK, *DHT11 Humidity & Temperature Sensor*, D-Robotics UK, 2010.

[8] R. O. Duda, P. E. Hart, and D. G. Stork, *Pattern Classification*, 2nd ed. New York: Wiley, 2001.

[9] STMicroelectronics, *L298 Dual Full-Bridge Driver*, STMicroelectronics, 2000.

[10] TowerPro, *SG90 Micro Servo*, TowerPro.

[11] Worldsemi Co., Ltd., *WS2812B Intelligent Control LED Integrated Light Source*, Worldsemi Co., Ltd.

[12] ON Semiconductor, *P2N2222A Amplifier Transistors NPN Silicon*, ON Semiconductor, 2013.

[13] T. D. Rossing, F. R. Moore, and P. A. Wheeler, *The Science of Sound*, 3rd ed. San Francisco: Addison Wesley, 2002.

[14] K. Ogata, *Ingeniería de control moderna*, 5th ed. Madrid: Pearson Educación, 2010.

[15] K. J. Åström and R. M. Murray, *Feedback Systems: An Introduction for Scientists and Engineers*. Princeton, NJ: Princeton University Press, 2008.

[16] C. Liechti, *pySerial Documentation*, accedido: oct. 2026. [Online]. Available: https://pyserial.readthedocs.io/

[17] J. D. Hunter, "Matplotlib: A 2D graphics environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007.